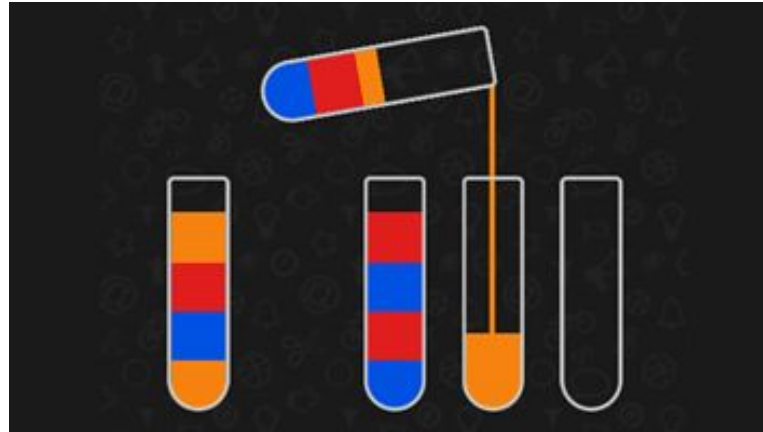


Assignment 1 - Checkpoint Delivery

Water Sort



André Cotrim up202305592
Dinis Noronha up202306120
João Vila Cova up202307756

Water Sort - Problem Overview

The Water Sort problem is a puzzle played on a set of bottles, each with a certain capacity. The bottles are initially filled with segments of colored water, stacked on top of one another.

The user can perform a move, pouring the top color segment from one bottle into another.

Goal: The puzzle is solved when every non-empty bottle contains exactly one color filling it completely.

Related Work

Book: Stuart Russel, Peter Norvig; Artificial Intelligence: A modern Approach.

Article: Sorting Balls and Water: Equivalence and Computational Complexity
<https://arxiv.org/pdf/2202.09495>

Web page: A* Algorithm Application for Solving the Water Sort Puzzle
[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/Makalah2025/Makalah-IF2211-Strategi-Algoritma-2025%20\(39\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/Makalah2025/Makalah-IF2211-Strategi-Algoritma-2025%20(39).pdf)

Web page: Python unit testing <https://www.dataquest.io/blog/unit-tests-python/>

Problem Formulation as a Search Problem

State representation: $S=(B_0,B_1,\dots,B_{T-1})$, where each B_i is an ordered list of color units (bottom $\rightarrow$ top), represented as integers:

$$B_i=(c_1,c_2,\dots,c_k), k \leq C, c \in \{1,\dots,N\}$$

Initial State: any arbitrary distribution of colors across bottles, having two bottles empty.

Goal State: $\forall i \in \{0, \dots, T-1\}: B_i = () \vee (|B_i| = C \wedge |\{c \in B_i\}| = 1)$

Every bottle is either empty, or full with exactly one color.

Problem Formulation as a Search Problem

Operations:

Name	Preconditions	Effect	Cost
Pour	$i \neq j \wedge i \text{ not empty} \wedge j \text{ not full} \wedge (j \text{ empty} \vee \text{top}(i) = \text{top}(j))$	pours until to j until (i empty $\wedge$ j full $\wedge$ top(i) $\neq$ top(j))	1

Heuristics functions:

- h1: Number of non-empty bottles that are not already solved.
- h2: Number of water segments in bottles that differ from bottle's top color
- h3: Mixture penalty plus incompleteness penalty, where mixture penalty is the number of color segments in all bottles, and incompleteness penalty is the number of bottles that are not empty or full with one color.
- h4: Number of color transitions (groups) inside each bottle. Counts groups of consecutive equal colors.

Implementation

Programming language: Python

GUI: Pygame

Architectural pattern: MVC (game, gui, search)

Unit testing: Python unit testing framework: unittest

Data structures: GameState (contains the state of the game and the bottles represented as list of colors) and TreeNode (holds a state of a problem for searching algorithms)